

PARTE I. Conociendo el proyecto

A Revisando línea base

1. El proyecto “miniTunes” contiene una construcción parcial del sistema. Revisen el directorio donde se encuentra el proyecto. Describan el contenido en términos de
 - a. Los directorios
El proyecto está contenido en un directorio llamado miniTunes cuenta por dentro con otro directorio llamado miniTunes en el cual tiene algunos archivos y otro directorio llamado doc y a su vez otro directorio interno que se llama resources
 - b. Las extensiones de los archivos
Dentro de la carpeta del proyecto está un .asta y una carpeta llamada miniTunes dentro de ella hay .class, .ctxt, .java, .bluej
2. Explore el proyecto en BlueJ
 - a. ¿Cuántas clases tiene?
Tiene tres clases
 - b. ¿Cuál es la relación entre ellas?
Relación de dependencia
 - c. ¿Cuál es la clase principal de la aplicación?
MiniTunes
 - d. ¿Cómo la reconocen?
Porque es la que ofrece los servicios de la aplicación es decir gestiona las operaciones que el usuario puede realizar sobre el sistema
 - e. ¿Cuál es la clase “diferente”?
Playlist
 - f. ¿Cuál es su propósito? Para las siguientes dos preguntas sólo consideren las clases “normales”:
Sirve para representar y gestionar individualmente una playlist y sus canciones.
3. Generen y revisen la documentación del proyecto: ¿está completa la documentación de cada clase? (Detallen el estado de documentación: encabezado y métodos)

La documentacion esta muy incompleta casi nula ya que lo unico que s esta haciendo es comentarios pero nunca se están agregando los encabezados `/** ... */` y ningun metodo incluyendo el constructor tienen documentación
4. Revisen el código fuente del proyecto, ¿en qué estado está cada clase? (Detallen el estado de las fuentes considerando dos dimensiones:
 - a. Atributos y métodos
En las clases PlayList y MiniTunes cuentan con métodos y atributos
 - b. código, documentación y comentarios
Todas las clases cuentan con código y comentarios sin embargo falta un poco de documentación ya que lo único es que miniTunes tiene encabezado sin embargo

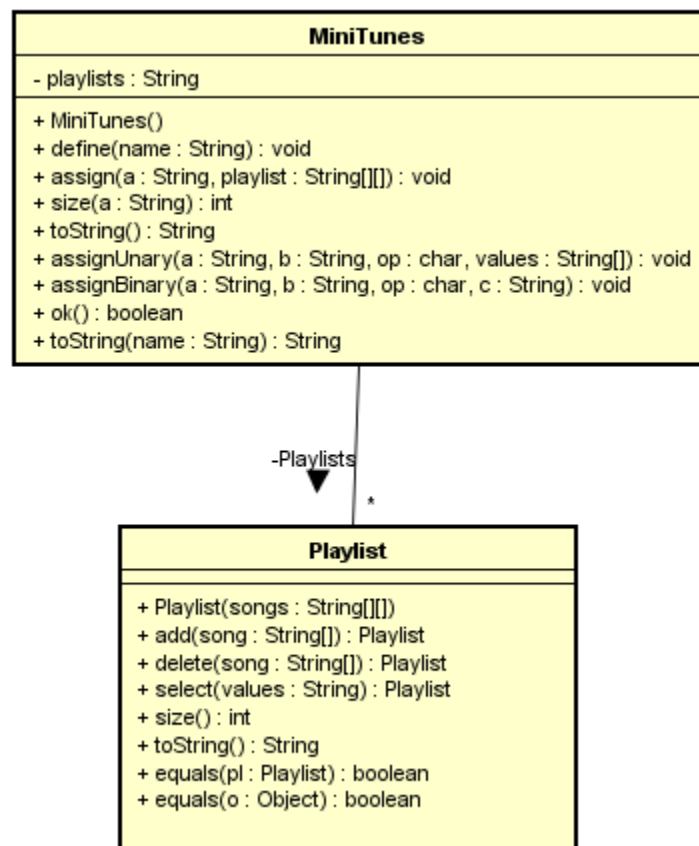
PlayList no cuenta con ello además en ambas clases falta la documentación de los métodos

- c. ¿Qué diferencia hay entre el código, la documentación y los comentarios?
El código son las instrucciones escritas en Java que ejecuta el programa; la documentación explica cómo funciona el código y cómo utilizar sus clases y métodos, por ejemplo mediante Javadoc; y los comentarios son anotaciones dentro del código que ayudan al programador a entenderlo, pero no afectan su ejecución.

B Realizando ingeniería inversa [En lab02.pdf miniTunes.astah]

MDD: MODEL DRIVEN DEVELOPMENT

1. Completen el diagrama de clases correspondiente al proyecto. (No incluyan la clase de pruebas)



2.

- a. ¿Cuáles contenedores están definidos?
Están definidos `TreeMap`, y en `PlayList` están `String[]` y `String[][]` que son arreglos
- b. ¿Qué diferencias hay entre el nuevo contenedor y el `ArrayList` y el vector `[]` que conocemos? Consulte el API de Java.

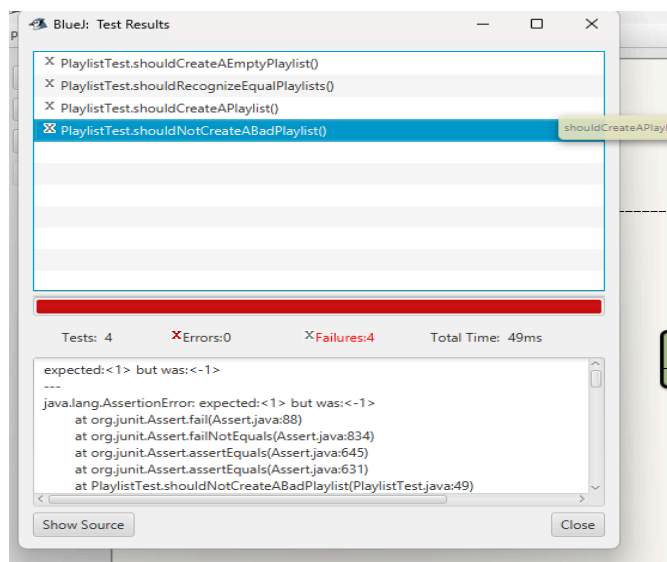
El TreeMap almacena elementos como pares clave-valor y permite buscarlos mediante una clave. El ArrayList almacena elementos en una lista de tamaño dinámico y se accede a ellos mediante índices. El vector [] también usa índices, pero tiene un tamaño fijo desde su creación.

3. En el nuevo contenedor,
 - a. ¿Cómo adicionamos un elemento?
Con put(clave, valor).
 - b. ¿Cómo lo consultamos?
Con get(clave).
 - c. ¿Cómo lo eliminamos?
Con remove(clave).

PARTE II. Aprendiendo a probar

A Conociendo Pruebas en BlueJ [En lab02.pdf *.java]

1. Revisen el código de la clase PlaylistTest
 - a. ¿cuáles etiquetas tiene (componentes con símbolo@)?
@Before, @Test, @After
¿cuántos métodos tiene?
Tiene 6 metodos
 - b. ¿cuantos métodos son de prueba?
Tiene 4 métodos de prueba
 - c. ¿cómo los reconocen?
Por la etiqueta @Test que aparece inmediatamente antes de cada método.
2. Ejecuten los tests de la clase PlaylistTest. (click derecho sobre la clase, Test All)
 - a. ¿cuántas pruebas se ejecutan?
Se ejecutan 4 test
 - b. ¿cuántas pasan?
Ninguna de las 4 pruebas pasa
 - c. ¿por qué?
Porque la clase playlist no está implementada
 - d. Capturen la pantalla.



3. Estudien las etiquetas encontradas en 1. Expliquen en sus palabras su significado.

- **@Before:** indica que ese método se ejecuta antes de cada prueba para preparar lo necesario.
- **@Test:** indica que el método es una prueba que debe ejecutar.
- **@After:** indica que ese método se ejecuta después de cada prueba para limpiar o finalizar lo utilizado.

4. Estudie los métodos `assertTrue`, `assertFalse`, `assertEquals`, `assertNull` y `fail` de la clase `Assert` del API JUnit 1. Explique en sus palabras que hace cada uno de ellos.

- `assertTrue`: verifica que la condición pasada sea verdadera, si la condición es false, la prueba falla, se usa cuando queremos comprobar que algo se cumple
- `assertFalse`: verifica que la condición pasada sea falsa, si la condición es true, la prueba falla, se usa cuando queremos comprobar que algo no debería ocurrir
- `assertEquals`: Compara dos valores (objetos o primitivos) y verifica que sean iguales, si falla muestra el valor esperado y el valor real para ayudar a depurar.
- `assertNull`: Verifica que el objeto sea nulo si falla Indica que el objeto no era nulo como se esperaba.

5. Investiguen y expliquen la diferencia entre un fallo y un error en Junit. Escriba código, usando los métodos del punto 4., para codificar los siguientes tres casos de prueba y lograr que se comporten como lo prometen `shouldPass`, `shouldFail`, `shouldErr`.

- **Fallo (Failure):** la prueba se ejecuta, pero el resultado no es el esperado.
- **Error (Error):** ocurre un problema durante la ejecución de la prueba, como una excepción o error en el código.

```
69  @Test
70  public void shouldPass(){
71      String [][] songs =
72          {{"One", "U2", "Rock", "4", "*****"},
73           {"Numb", "Linkin Park", "Rock", "3", null}};
74
75      Playlist pl = new Playlist(songs);
76
77      assertTrue(pl.size() > 0);
78      assertFalse(pl.size() == 0);
79      assertEquals(2, pl.size());
80      assertNull(songs[1][4]);
81  }
82
83  @Test
84  public void shouldFail(){
85      String [][] songs =
86          {{"One", "U2", "Rock", "4", "*****"},
87           {"Numb", "Linkin Park", "Rock", "3", null}};
88
89      Playlist pl = new Playlist(songs);
90      assertEquals(5, pl.size());
91  }
92
93  @Test
94  public void shouldErr(){
95      String [][] songs = {};
96      Playlist pl = new Playlist(songs);
97      assertTrue(pl.size() >= 0);
98      String song = songs[0][0];
99  }
```

B Practicando Pruebas

Ahora vamos a escribir el código necesario para que las pruebas de pasen PlaylistTest.

1.
 - a. Determinen los atributos de la clase Playlist.
Vamos a usar un `ArrayList<String[]>`
 - b. Justifiquen la selección.
Necesitamos un arreglo dinámico por esto usamos `ArraList` y además este va a hacer un arreglo de listas de Strings ya que es así como viene la información
2.
 - a. Determinen el invariante de la clase Playlist.
El invariante de Playlist establece que todas las canciones almacenadas tienen un título y un artista definidos, no existen dos canciones con la misma combinación de título y artista, la duración, cuando está definida, está entre 1 y 9 minutos, y el rating, cuando está definido, está entre * y *****.
 - b. Justifiquen la decisión.
Por las reglas de negocio que aparecen en el comentario fijado. Esto va a hacer así ya que es la estructura de como va a estar guardando siempre y nunca va a cambiar
3. Implementen los métodos de Playlist necesarios para pasar todas las pruebas definidas.
 - a. ¿Cuáles métodos implementaron?
Los métodos constructores, `add()`, `normalizar()`, `check()`, `size()`, `equals(Object o)`, `equals(Playlist pl)`
4. Capturen los resultados de las pruebas de unidad.

